CSBAD22ET2 Big Data Analytics Lab | ETL

Ex.No.	Content	Page No
1	Exploration of Installation of HADOOP	01
2	Implementation of File Management Task	10
3	Basic Word Count Map Reduce program	15
4	Install, Configure and run Python, Numpy & Pandas	23
5	Visualize Data using Basic Plotting Techniques in Python	37

P. Dineshkumar
Dept of CSE
96771 43550

EXP NO:01

EXPLORATION OF INSTALLTION OF HADOOP

Exp : 01

AIM: To install and setup Hadoop environment in windows

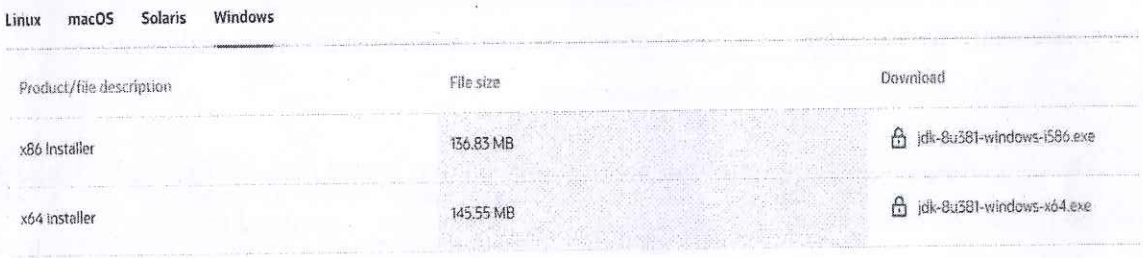
ALGORITHM:

STEP 1: To install Hadoop the primary task is to setup and install java environment

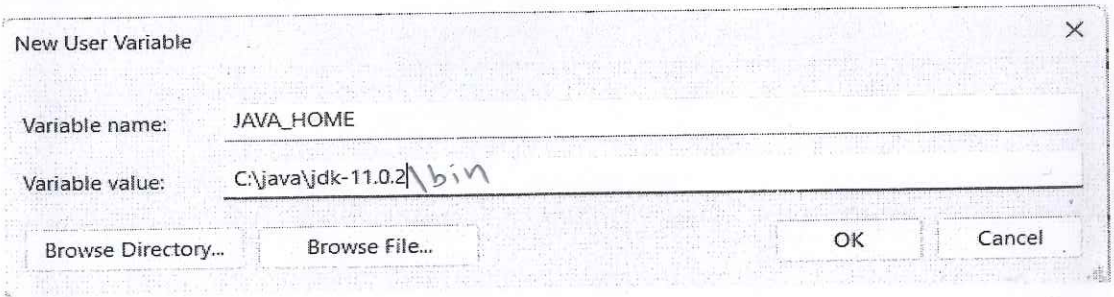
STEP 2: The java version that needed to be installed depends on the Hadoop's version. Here we are installing the latest version of Hadoop which is 3.3.0 which supports java version varying 8 -11 from 8-11(runtime only).

STEP 3: Use the following link to install java

https://www.oracle.com/java/technologies/downloads/#java8-windows



STEP 4: After installing java setup, the java environment in environmental variables directing the bin folder inside the java folder (**C:\java\jdk-11.0.2\bin**) copy the path till bin folder and paste it in the environmental variable define the new path and add the bin folder location as **JAVA_HOME=" C:\java\jdk11.0.2\bin"** and apply the changes



STEP 5: Now after setting up the java environment check the setup has been successfully set by using **java -version** command in your command prompt and it should display the version of java you have installed.

```
C:\Windows\System32>java -version
java version "1.8.0_381"
Java(TM) SE Runtime Environment (build 1.8.0_381-b09)
Java HotSpot(TM) 64-Bit Server VM (build 25.381-b09, mixed mode)
```

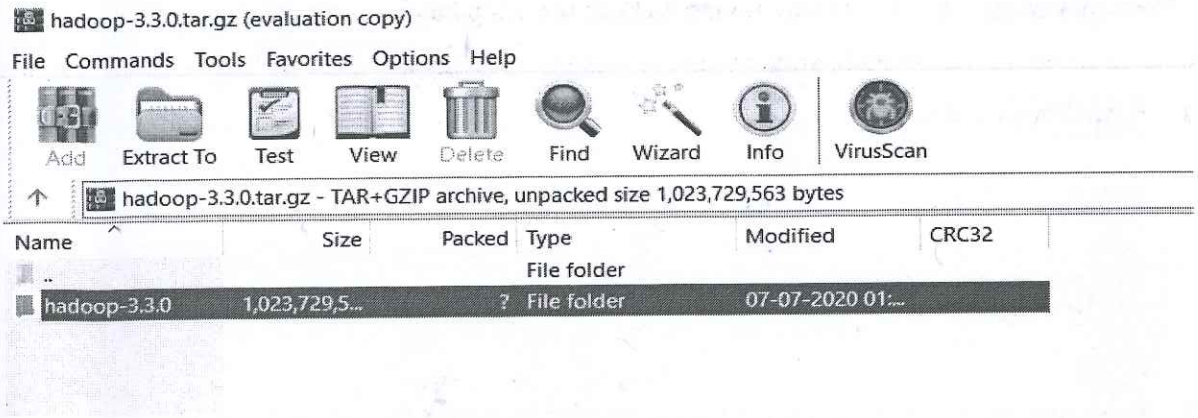
STEP 7: Hadoop is Unix distribution-based file with tar.gz extension we have to extract the file using the 7-zip manager which supports multiple formats follow this link to install 7-zip https://7-zip.org/

STEP 8: Now install the Notepad++ text editor which is further used to modify or edit the configuration file within Hadoop as per our requirement

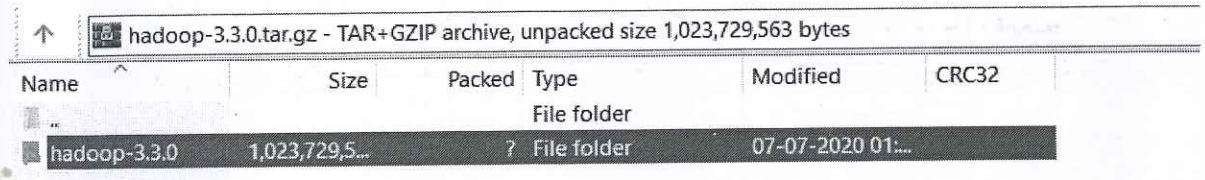
STEP 9: After installing and setting up all the required application install Hadoop from the official Apache Hadoop website https://hadoop.apache.org/releases.html download the binary download which can run directly without any need for compilation.

Version	Release date	Source download	Binary download	Release notes
3.3.6	2023 Jun 23	source (checksum signature)	binary (checksum signature) binary-aarch64 (checksum signature)	Announcement
3.2.4	2022 Jul 22	source (checksum signature)	binary (checksum signature)	Announcement
2.10.2	2022 May 31	source (checksum signature)	binary (checksum signature)	Announcement

STEP 10: Run 7-zip manager as administrator and navigate to the path where Hadoop is located for extract the compiled binary download of Hadoop



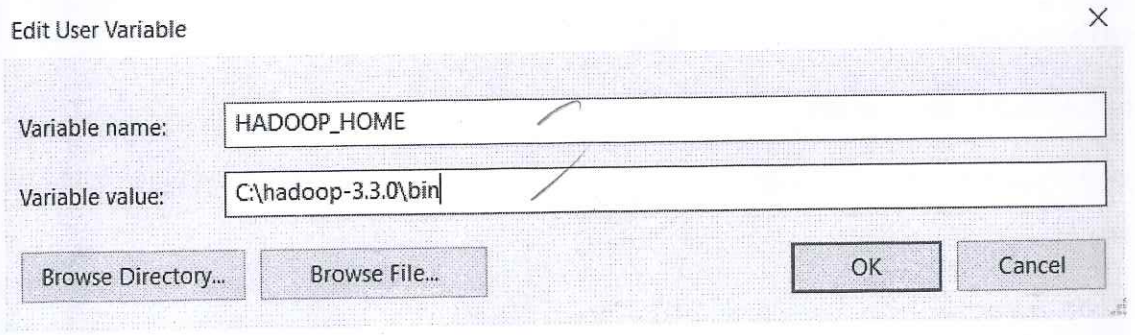
STEP 11: After doing the extraction process there is another compressed file with in the extracted file extract that as well.



STEP 12: From the extracted folder replace the bin file with the reliable windows supported configured file here is the drive link to download the bin file

https://drive.google.com/file/d/1kVhX9snOZ3oLUxDjh3AVI8fcRnEWAAE4/view

STEP 13: Setup the Hadoop environment in environment variable and set path location as **HADOOP_HOME= "C:\ hadoop-3.3.0\bin"**



STEP 14: Add the Hadoop bin and sbin path location by editing the path. And add the bin, sbin location there

P.DINESH KUMAR
Dept.of CSE
Dr.MGR UNIVERSITY
Cell:96771 43550



STEP 15: Now open etc folder inside the Hadoop folder and locate the file Hadoop-env.cmd and set the java home location

```
@rem remote nodes.

@rem The java implementation to use.  Required.
set JAVA_HOME=%JAVA_HOME%
set JAVA_HOME=C:\java\java8

@rem The jsvc implementation to use. Jsvc is required to run secure datanodes.
@rem set JSVC_HOME=%JSVC_HOME%

@rem set HADOOP_CONF_DIR=
```

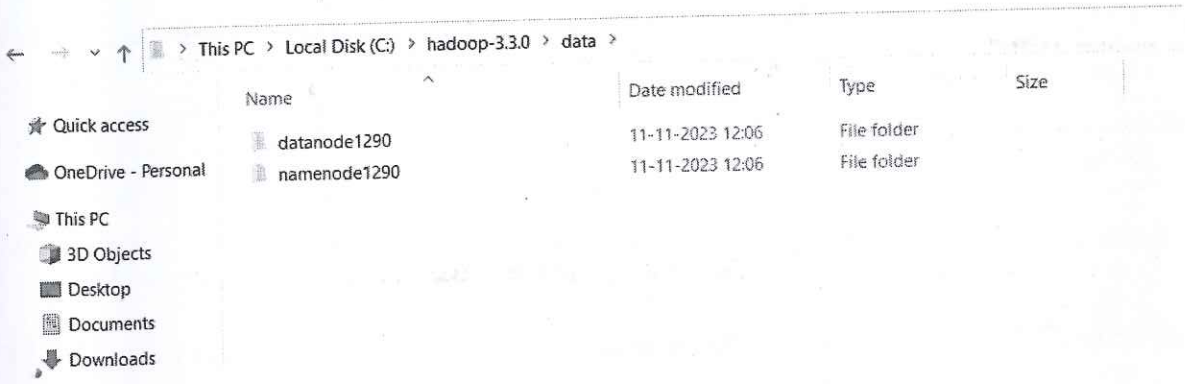
STEP 16: Edit the following configuration XML files core-site.xml, hdfs-site.xml, mapred-site.xml, yarn-site.xml are used to configure the behaviour of your Hadoop Cluster and save them.

STEP 17: Starting from core-site.xml edit it using notepad++ and following the program to configure.

PROGRAM (CORE-SITE.XML)

```
<configuration>
 <property>
 <name>fs.default.name</name>
 <value>hdfs://localhost:9000</value>
 </property>
 </configuration>
```

STEP 18: To edit the hdfs-site.xml create Date folder and then within the Data folder create Namenode, Datenode which are used to manage and cluster flow date and log files.



STEP 19: Now open the hdfs-site.xml in notepad++ and add the following program

```
<configuration>

<property>

<name>dfs.replication</name>

<value>1</value>

</property>

<property>

<name>dfs.namenode.name.dir</name>

<value> C:/hadoop-3.3.0/Data/datanode1290 </value>

</property>

<property>

<name>dfs.datanode.data.dir</name>
```

<value>> C:/hadoop-3.3.0/Data/namenode1290 </value>

</property>

</configuration>

STEP 20: Edit mapred-site.xml file using notepad++ add this following program

<configuration>

<property>

<name>mapreduce.framework.name</name>

<value>yarn</value>

</property>

</configuration>

STEP 21: Edit the yarn-site.xml with following program and save it.

<configuration>

<property>

<name>yarn.nodemanager.aux-services</name>

<value>mapreduce_shuffle</value>

</property>

<property>

<name>yarn.nodemanager.auxservices.mapreduce.shuffle.class</name>

<value>org.apache.hadoop.mapred.ShuffleHandler</value>

</property>

</configuration>

STEP 22: Now save them and open command prompt as administrator and run the following command to **hdfs namenode -format** to format the contents of namenode

```
\current\fsimage.ckpt_0000000000000000000 using no compression
2023-11-06 11:45:46,760 INFO namenode.FSImageFormatProtobuf: Image file C:\hadoop-3.3.0\data\namenode1290\curren
t\fsimage.ckpt_0000000000000000000 of size 400 bytes saved in 0 seconds .
2023-11-06 11:45:46,783 INFO namenode.NNStorageRetentionManager: Going to retain 1 images with txid >= 0
2023-11-06 11:45:46,794 INFO namenode.FSImage: FSImageSaver clean checkpoint: txid=0 when meet shutdown.
2023-11-06 11:45:46,795 INFO namenode.NameNode: SHUTDOWN_MSG:
/************************************************************
SHUTDOWN_MSG: Shutting down NameNode at Azhar/127.0.0.1
************************************************************/

C:\Windows\system32>
```

STEP 23: To check the daemons configured correctly open command prompt as administrator and run the following command's

hdfs namenode, hdfs Datanode, yarn nodemanager, yarn resourcemanager

hdfs namenode

```
C:\hadoop-3.3.0\sbin>hdfs namenode
2023-11-06 11:47:10,614 INFO namenode.NameNode: STARTUP_MSG:
/************************************************************
STARTUP_MSG: Starting NameNode
STARTUP_MSG:   host = Azhar/127.0.0.1
STARTUP_MSG:   args = []
STARTUP_MSG:   version = 3.3.0
STARTUP_MSG:   classpath = C:\hadoop-3.3.0\etc\hadoop;C:\hadoop-3.3.0\share\hadoop\common;C:\hadoop-3.3.0\share\
hadoop\common\lib\accessors-smart-1.2.jar;C:\hadoop-3.3.0\share\hadoop\common\lib\animal-sniffer-annotations-1.1
```

hdfs datanode

```
C:\Windows\system32>hdfs datanode
2023-11-06 11:51:47,958 INFO datanode.DataNode: STARTUP_MSG:
/************************************************************
STARTUP_MSG: Starting DataNode
STARTUP_MSG:   host = Azhar/127.0.0.1
STARTUP_MSG:   args = []
STARTUP_MSG:   version = 3.3.0
STARTUP_MSG:   classpath = C:\hadoop-3.3.0\etc\hadoop;C:\hadoop-3.3.0\share\hadoop\common;C:\hadoop-3.
3.0\share\hadoop\common\lib\accessors-smart-1.2.jar;C:\hadoop-3.3.0\share\hadoop\common\lib\animal-sni
ffer-annotations-1.17.jar;C:\hadoop-3.3.0\share\hadoop\common\lib\asm-5.0.4.jar;C:\hadoop-3.3.0\share\
```

yarn nodemanager

```
C:\Windows\system32>yarn nodemanager
2023-11-06 11:52:28,180 INFO nodemanager.NodeManager: STARTUP_MSG:
/************************************************************
STARTUP_MSG: Starting NodeManager
STARTUP_MSG:   host = Azhar/127.0.0.1
STARTUP_MSG:   args = []
STARTUP_MSG:   version = 3.3.0
STARTUP_MSG:   classpath = C:\hadoop-3.3.0\etc\hadoop;C:\hadoop-3.3.0\etc\hadoop;C:\hadoop-3.3.0\etc\h
adoop;C:\hadoop-3.3.0\share\hadoop\common;C:\hadoop-3.3.0\share\hadoop\common\lib\accessors-smart-1.2.
jar;C:\hadoop-3.3.0\share\hadoop\common\lib\animal-sniffer-annotations-1.17.jar;C:\hadoop-3.3.0\share\
```

yarn resourcemanager

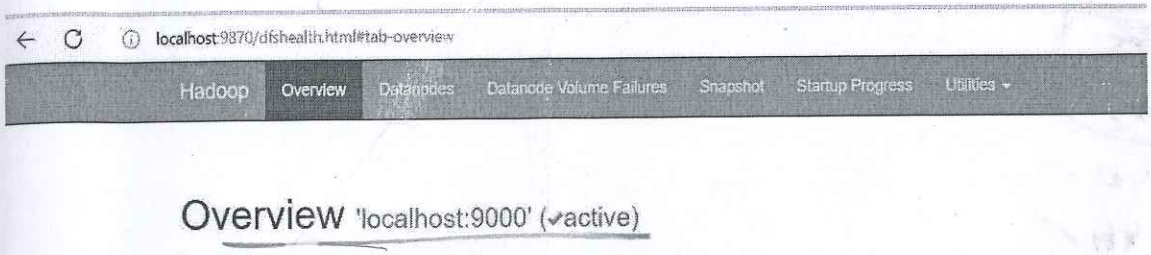
```
C:\Windows\system32>yarn resourcemanager
2023-11-06 11:53:32,487 INFO resourcemanager.ResourceManager: STARTUP_MSG:
/************************************************************
STARTUP_MSG: Starting ResourceManager
STARTUP_MSG:   host = Azhar/127.0.0.1
STARTUP_MSG:   args = []
STARTUP_MSG:   version = 3.3.0
STARTUP_MSG:   classpath = C:\hadoop-3.3.0\etc\hadoop;C:\hadoop-3.3.0\etc\hadoop;C:\hadoop-3.3.0\etc\h
adoop;C:\hadoop-3.3.0\share\hadoop\common;C:\hadoop-3.3.0\share\hadoop\common\lib\accessors-smart-1.2.
jar;C:\hadoop-3.3.0\share\hadoop\common\lib\animal-sniffer-annotations-1.17.jar;C:\hadoop-3.3.0\share\
```

STEP 24: To check the check the daemons that are running in background we can use Java Virtual Machine Process Status which is used to list the java virtual machines that are currently running on a system it is used to display the process ID (PID) of each JVM

```
C:\Windows\System32>jps
9712 NodeManager
8212 Jps
16056 NameNode
1800 ResourceManager
18200 DataNode
```

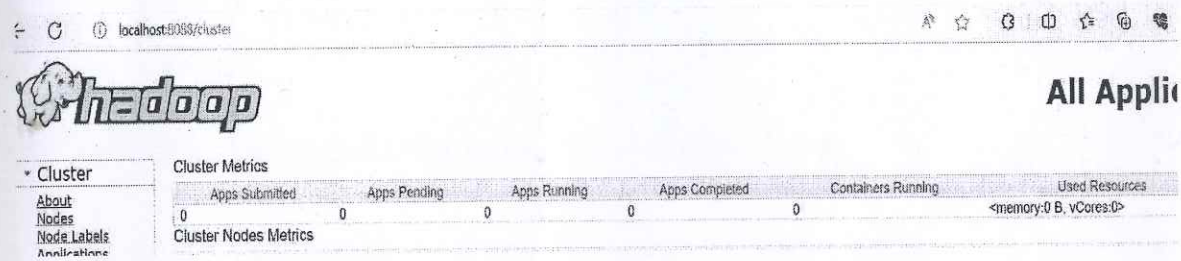
STEP 25: Now we can access the Namenode and Datanode as web user interface (web-UI) by using the following localhost address

localhost:9870



STEP 26: To access the Datanode use the following localhost address

localhost:8088



RESULT: Thus, the program to install and setup Hadoop has been completed and Output is verified successfully completed.

P.DINESH KUMAR
Dept.of CSE
Dr.MGR UNIVERSITY
Cell:96771 43550

EXP.NO:02

IMPLEMENTATION OF FILE MANAGEMENT TASKS

Exp : 02

AIM: To implement file management tasks in Hadoop file system (HDFS).

ALGORITHM:

1. **Creating a directory in HDFS**

SYNTAX:

hadoop fs -mkdir <paths>

EXAMPLE:

hadoop fs -mkdir /user

hadoop fs -mkdir /user/dir1

```
C:\Windows\System32>hadoop  fs -mkdir /user

C:\Windows\System32>hadoop fs -mkdir /user/dir1
```

2. **Listing the contents of a directory**

SYNTAX:

hadoop fs -ls <directory name>

EXAMPLE:

hadoop fs -ls /user

hadoop fs -ls /user/dir1

```
C:\Windows\System32>hadoop fs -ls /user
Found 1 items
drwxr-xr-x   - moham supergroup          0 2023-11-05 08:47 /user/dir1

C:\Windows\System32>hadoop fs -ls /user/dir1
```

3. Uploading and downloading a file in HDFS

SYNTAX: (UPLOAD)

hadoop fs -put <local file system path> <hdfs destination path>

EXAMPLE:

hadoop fs -put C:\Home\samplefile.txt.txt /user/dir1/

hadoop fs -ls /user/dir1

```
C:\Windows\System32>hadoop fs -put C:\Home\samplefile.txt /user/dir1

C:\Windows\System32>hadoop fs -ls /user/dir1
Found 1 items
-rw-r--r--   1 moham supergroup         13 2023-11-05 14:46 /user/dir1/samplefile.txt
```

SYNTAX: (DOWNLOAD)

hadoop fs -get <hdfs src> <local dst>

EXAMPLE:

hadoop fs -get /user/dir1/samplefile.txt C:\Home\Hadoopfiles

```
C:\Windows\System32>hadoop fs -get /user/dir1/samplefile.txt C:\Home\Hadoopf
```

4. See the contents of a file

SYNTAX:

hadoop fs -cat <path[filename]>

EXAMPLE:

hadoop fs -cat /user/dir1/samplefile.txt

```
C:\Windows\System32>hadoop fs -cat /user/dir1/samplefile.txt
1
2
3
4
5
```

5. Copy a file from source to destination

SYNTAX:

hadoop fs -cp <src> <dst>

EXAMPLE:

hadoop fs /user/dir1/samplefile.txt /user/dir2

```
C:\Windows\System32>hadoop fs -cp /user/dir1/samplefile.txt /user/dir2

C:\Windows\System32>hadoop fs -ls /user/dir2
Found 1 items
-rw-r--r--   1 moham supergroup         13 2023-11-05 14:57 /user/dir2/samplefile.txt
```

6. Copy a file from and to local file system to hdfs

SYNTAX: (FROM)

hadoop fs -copyFromLocal <local file system file path> <hdfs dst>

EXAMPLE:

hadoop fs -copyFromLocal C:\Home\test.txt /user/dir1

```
C:\Windows\System32>hadoop fs -copyFromLocal C:\Home\test.txt /user/dir1

C:\Windows\System32>hadoop fs -ls /user/dir1
Found 2 items
-rw-r--r--   1 moham supergroup         13 2023-11-05 14:46 /user/dir1/samplefile.txt
-rw-r--r--   1 moham supergroup          0 2023-11-05 15:00 /user/dir1/test.txt
```

SYNTAX: (TO)

hadoop fs -copyToLocal <hdfs src> <local dst>

EXAMPLE:

hadoop fs -copyToLocal /user/dir1/samplefile.txt C:\Home\copy

```
C:\Windows\System32>hadoop fs -copyToLocal /user/dir1/samplefile.txt C:\Home\copy
```

7. Move file from source to destination

SYNTAX:

```
hadoop fs -mv <src> <dst>
```

EXAMPLE:

hadoop fs -mv /user/dir1/test.txt /user/dir2

```
C:\Windows\System32>hadoop fs -ls /user/dir2
Found 2 items
-rw-r--r--   1 moham supergroup         13 2023-11-05 14:57 /user/dir2/samplefile.t
-rw-r--r--   1 moham supergroup          0 2023-11-05 15:00 /user/dir2/test.txt
```

8. Remove a file or directory in hdfs

SYNTAX:

```
hadoop fs -rm <arg>
```

EXAMPLE:

hadoop fs -rm /user/dir1/samplefile.txt

```
C:\Windows\System32>hadoop fs -rm /user/dir1/samplefile.txt
Deleted /user/dir1/samplefile.txt
```

SYNTAX: (Recursive method for deleting directories)

```
hadoop fs -rm -r <arg>
```

EXAMPLE:

hadoop fs -rm -r /user/dir1

```
C:\Windows\System32>hadoop fs -rm -r /user/dir1
Deleted /user/dir1

C:\Windows\System32>hadoop fs -ls  /user
Found 1 items
drwxr-xr-x   - moham supergroup          0 2023-11-05 15:05 /user/dir2
```

9. Display few lines of a file

SYNTAX:

hadoop fs -tail <path[filename]>

EXAMPLE:

hadoop fs -tail /user/dir2/samplefile.txt

```
C:\Windows\System32>hadoop fs -tail /user/dir2/samplefile.txt
1
2
3
4
5
```

10. Display the aggregate length of a file

SYNTAX:

hadoop fs -du <path>

EXAMPLE:

hadoop fs -du /user/dir2/samplefile.txt

```
C:\Windows\System32>hadoop fs -du /user/dir2/samplefile.txt
17  17  /user/dir2/samplefile.txt
```

RESULT: Thus, the implementation of file management tasks in hadoop file system was implemented and Output is verified successfully.

EXP.NO: 03

BASIC WORD COUNT MAP REDUCE PROGRAM

Exp: 03

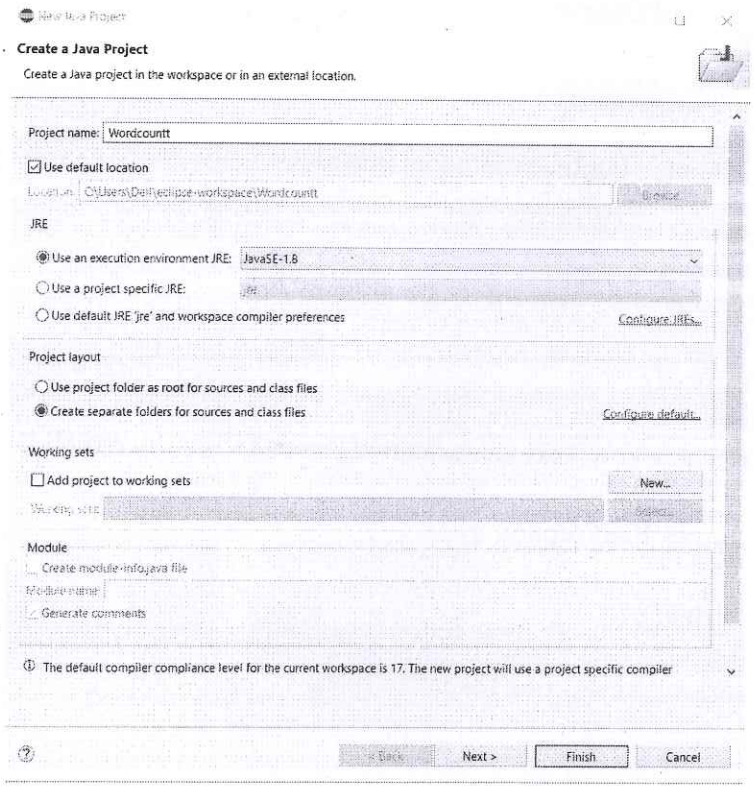
AIM: To Run a Basic Word Count Map Reduce program to understand Map Reduce Paradigm.

ALGORITHM:

STEP 1: Run Eclipse for Java Developers

STEP 2: Create a new Java Project with name "WordCount "

STEP 3: Set the Java Environment Version to your current version of Java (JRE : 1.8)



STEP 4 : Add a Package with name "com.mapreduce.java" and Create three Classes in it.

STEP 5 : Create a New Class With name WC_Mapper.java.

STEP 6: Now write the below program in the "WC_Mapper.java" Class

PROGRAM:

```
package com.mapreduce.java;

import java.io.IOException;

import java.util.StringTokenizer;

import org.apache.hadoop.io.IntWritable;

import org.apache.hadoop.io.LongWritable;

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapred.MapReduceBase;

import org.apache.hadoop.mapred.Mapper;

import org.apache.hadoop.mapred.OutputCollector;

import org.apache.hadoop.mapred.Reporter;

public class WC_Mapper extends MapReduceBase implements
Mapper<LongWritable,Text,Text,IntWritable>{

  private final static IntWritable one = new IntWritable(1);

  private Text word = new Text();

  public void map(LongWritable key, Text value,OutputCollector<Text,IntWritable> output

    Reporter reporter) throws IOException{

  String line = value.toString();

  StringTokenizer tokenizer = new StringTokenizer(line);

  while (tokenizer.hasMoreTokens()){

    word.set(tokenizer.nextToken());

    output.collect(word, one);

  }}

}
```

STEP 7: Now Create another class with name "WC_Reducer.java" and paste the below program in it.

PROGRAM:

```
package com.mapreduce.java;

import java.io.IOException;

import java.util.Iterator;

import org.apache.hadoop.io.IntWritable;

import org.apache.hadoop.io.Text;

import  org.apache.hadoop.mapred.MapReduceBase;

import   org.apache.hadoop.mapred.OutputCollector;

import org.apache.hadoop.mapred.Reducer;

import org.apache.hadoop.mapred.Reporter;

public class WC_Reducer extends MapReduceBase implements
Reducer<Text,IntWritable,Text,IntWritable>
{

public void reduce(Text key, Iterator<IntWritable> values,OutputCollector<Text,IntWritable> output,

 Reporter reporter) throws IOException
{

int sum=0;

while (values.hasNext())
{

sum+=values.next().get();

}

output.collect(key,new IntWritable(sum));

}

}
```

STEP 8: Now, Create another class with name "WC_runner.java" and paste the below program in it.

PROGRAM:

```
package com.mapreduce.java;

import java.io.IOException;

import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapred.FileInputFormat;

import org.apache.hadoop.mapred.FileOutputFormat;

import org.apache.hadoop.mapred.JobClient;

import org.apache.hadoop.mapred.JobConf;

import org.apache.hadoop.mapred.TextInputFormat;

import org.apache.hadoop.mapred.TextOutputFormat;

public class WC_Runner
{
  public static void main(String[] args) throws IOExceptior
  {
    JobConf conf = new JobConf(WC_Runner.class);

    conf.setJobName("WordCount");

    conf.setOutputKeyClass(Text.class);

    conf.setOutputValueClass(IntWritable.class);

    conf.setMapperClass(WC_Mapper.class);

    conf.setCombinerClass(WC_Reducer.class);

    conf.setReducerClass(WC_Reducer.class);

    conf.setInputFormat(TextInputFormat.class);
```

```
        conf.setOutputFormat(TextOutputFormat.class);

        FileInputFormat.setInputPaths(conf,new Path(args[0]));

        FileOutputFormat.setOutputPath(conf,new Path(args[1]));

        JobClient.runJob(conf);

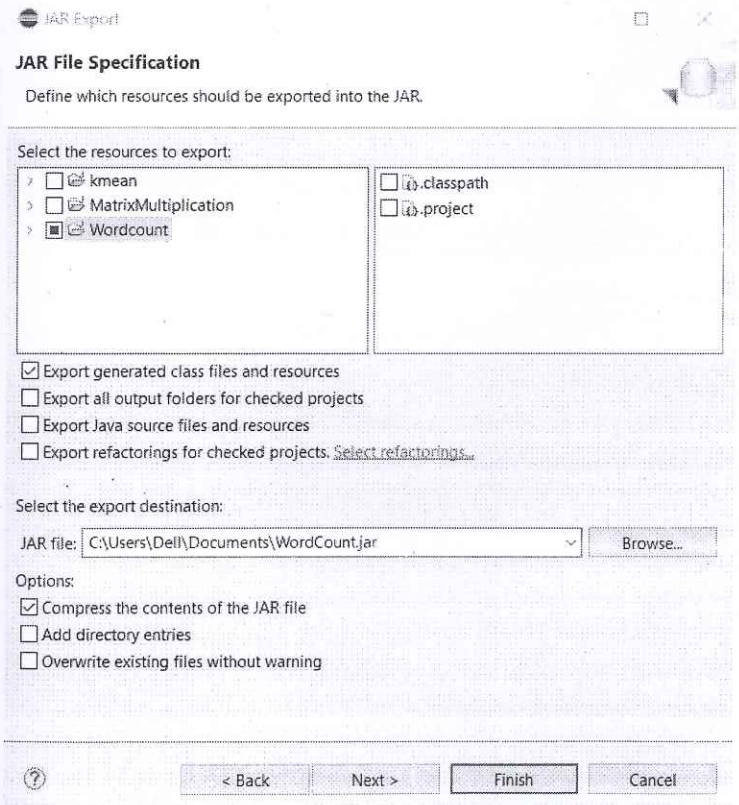
    }

}
```

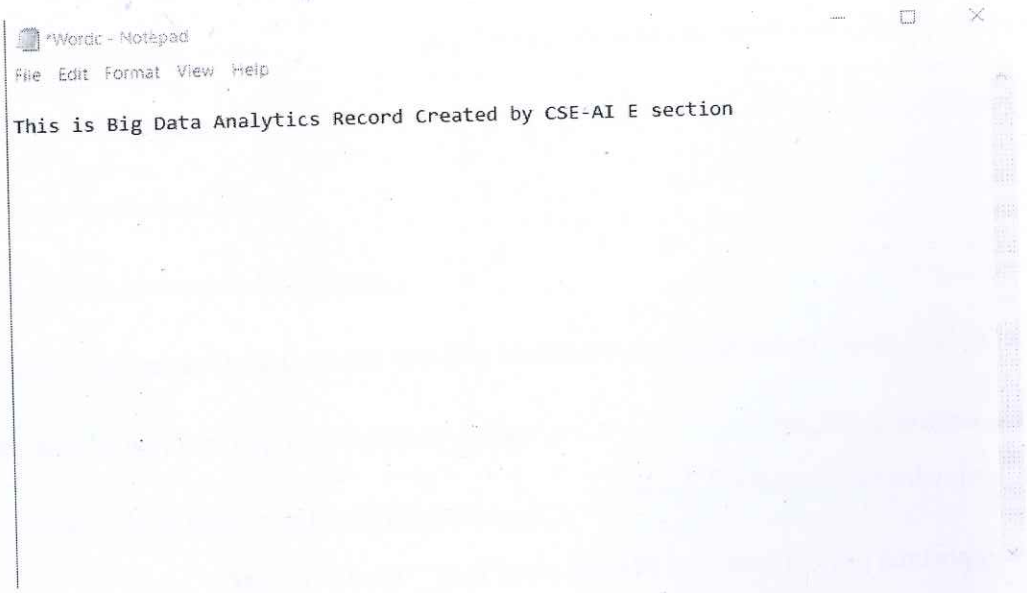
STEP 9: To resolve the errors in the programs we should add two External jar files to it.

- Hadoop_common :2.7.3.jar
- Hadoop_mapreduce:client:core:2.7.1.jar

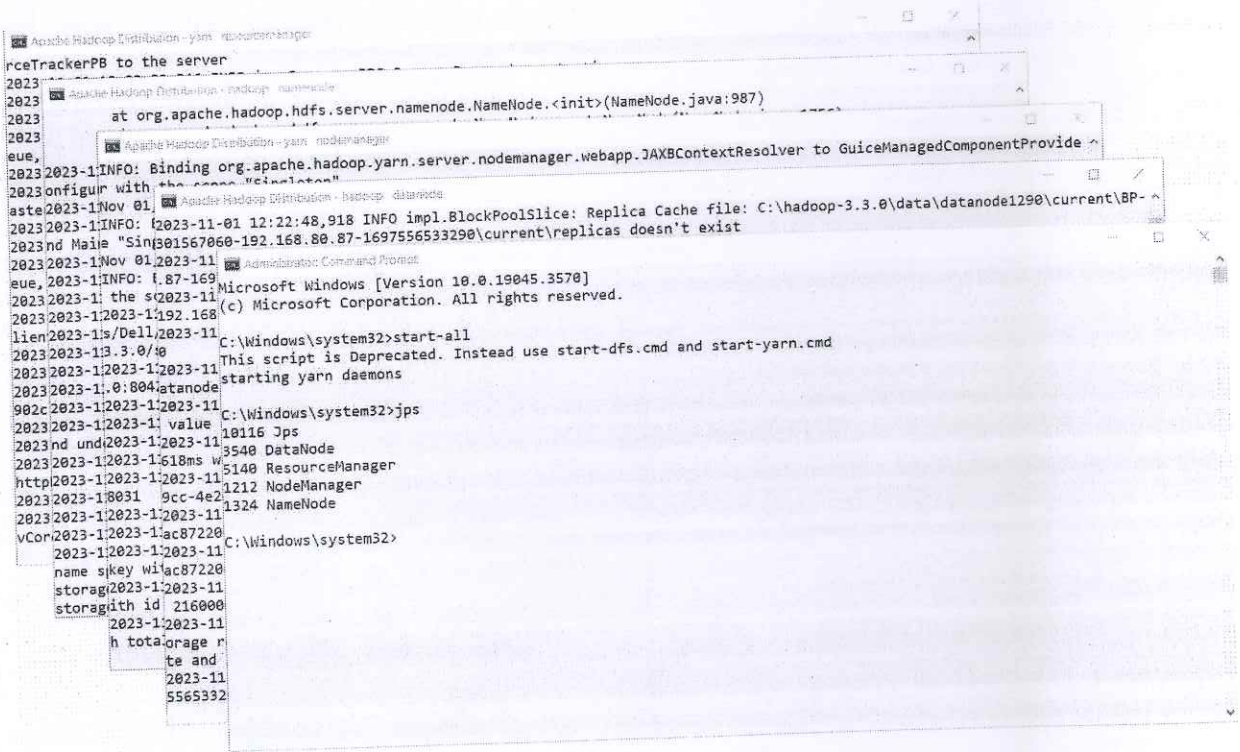
STEP 10: Now export the project into a Jar file and name it as "WordCount.jar"



STEP 11: Now create a Text file in Notepad and name it as "wordc.txt" and write some content inside the text file and save it.



STEP 12: Now run all the demons in Hadoop.

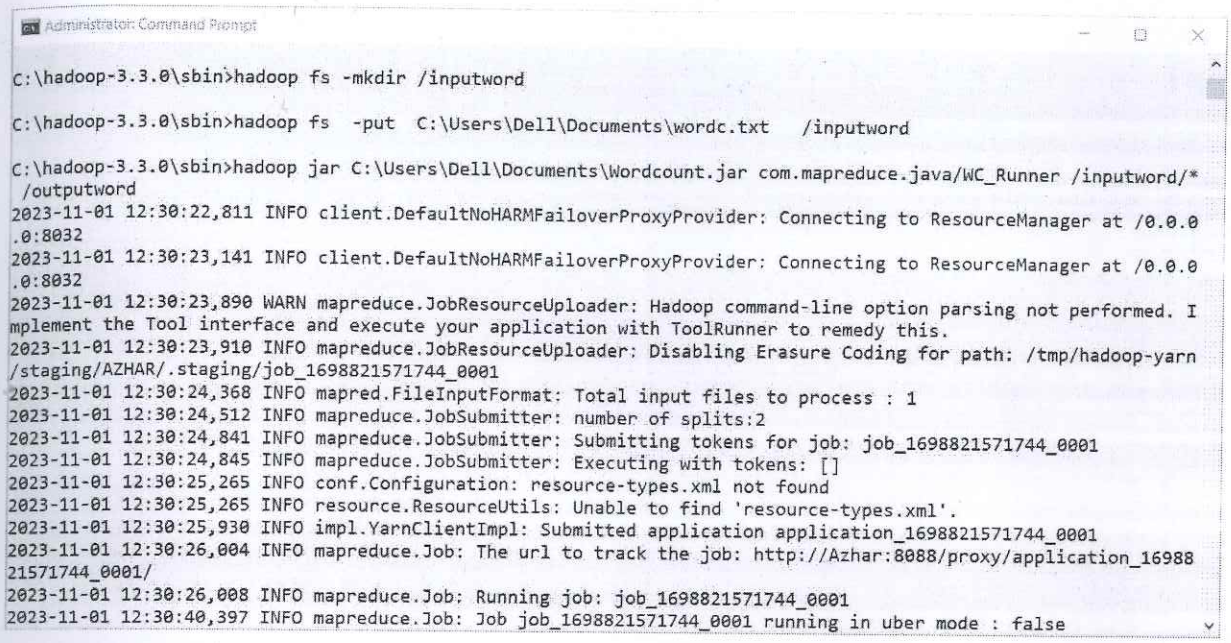


STEP 13: Create a new input directory named as "inputword".

By using the command: **hadoop fs -mkdir /inputword**

STEP 14: Now put the "wordc.txt" file to the inputword directory.

By using the command: **hadoop fs -put C:\Users\Dell\Documents\wordc.txt /inputword**



```
Administrator: Command Prompt

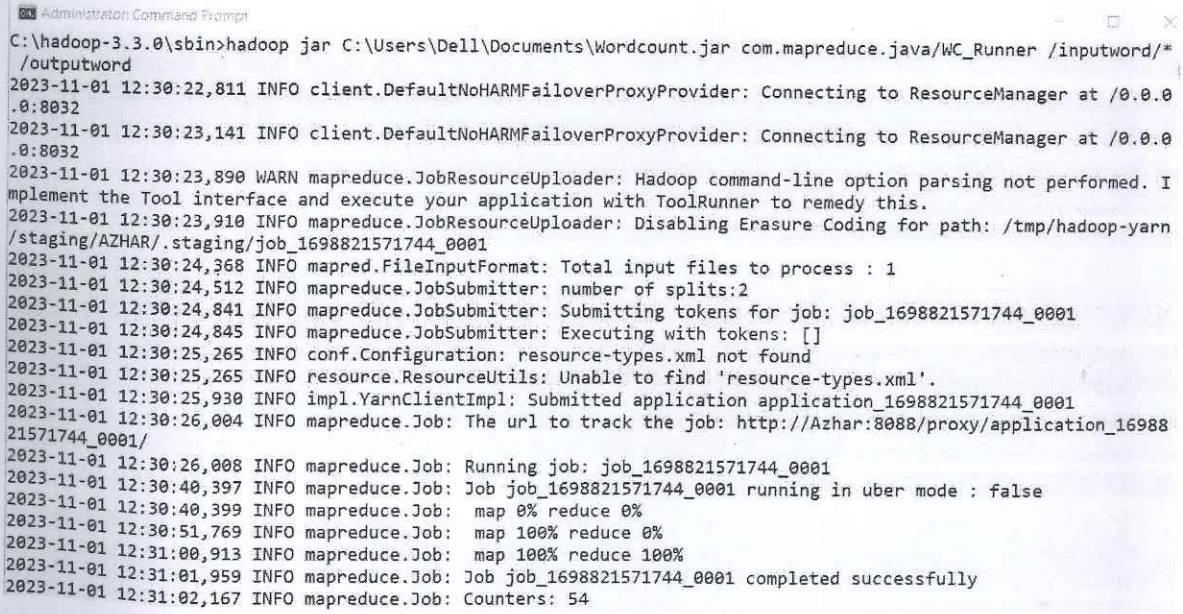
C:\hadoop-3.3.0\sbin>hadoop fs -mkdir /inputword

C:\hadoop-3.3.0\sbin>hadoop fs  -put  C:\Users\Dell\Documents\wordc.txt   /inputword

C:\hadoop-3.3.0\sbin>hadoop jar C:\Users\Dell\Documents\Wordcount.jar com.mapreduce.java/WC_Runner /inputword/*
 /outputword
2023-11-01 12:30:22,811 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0
.0:8032
2023-11-01 12:30:23,141 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0
.0:8032
2023-11-01 12:30:23,890 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. I
mplement the Tool interface and execute your application with ToolRunner to remedy this.
2023-11-01 12:30:23,910 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn
/staging/AZHAR/.staging/job_1698821571744_0001
2023-11-01 12:30:24,368 INFO mapred.FileInputFormat: Total input files to process : 1
2023-11-01 12:30:24,512 INFO mapreduce.JobSubmitter: number of splits:2
2023-11-01 12:30:24,841 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1698821571744_0001
2023-11-01 12:30:24,845 INFO mapreduce.JobSubmitter: Executing with tokens: []
2023-11-01 12:30:25,265 INFO conf.Configuration: resource-types.xml not found
2023-11-01 12:30:25,265 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2023-11-01 12:30:25,930 INFO impl.YarnClientImpl: Submitted application application_1698821571744_0001
2023-11-01 12:30:26,004 INFO mapreduce.Job: The url to track the job: http://Azhar:8088/proxy/application_16988
21571744_0001/
2023-11-01 12:30:26,008 INFO mapreduce.Job: Running job: job_1698821571744_0001
2023-11-01 12:30:40,397 INFO mapreduce.Job: Job job_1698821571744_0001 running in uber mode : false
```

STEP 15: Run the Jar file created from the project

Using the command: **hadoop jar C:\Users\Dell\Documents\Wordcount.jar com.mapreduce.java/WC_Runner /inputword/* /outputword**

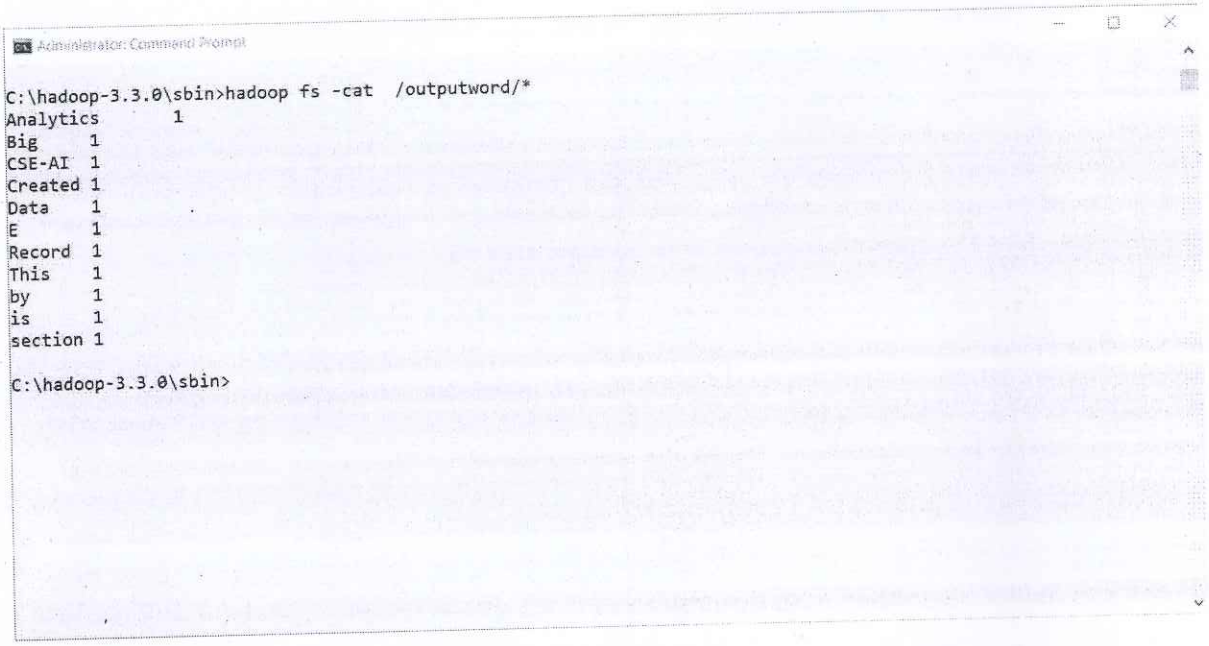


```
Administrator: Command Prompt
C:\hadoop-3.3.0\sbin>hadoop jar C:\Users\Dell\Documents\Wordcount.jar com.mapreduce.java/WC_Runner /inputword/*
 /outputword
2023-11-01 12:30:22,811 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0
.0:8032
2023-11-01 12:30:23,141 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0
.0:8032
2023-11-01 12:30:23,890 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. I
mplement the Tool interface and execute your application with ToolRunner to remedy this.
2023-11-01 12:30:23,910 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn
/staging/AZHAR/.staging/job_1698821571744_0001
2023-11-01 12:30:24,368 INFO mapred.FileInputFormat: Total input files to process : 1
2023-11-01 12:30:24,512 INFO mapreduce.JobSubmitter: number of splits:2
2023-11-01 12:30:24,841 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1698821571744_0001
2023-11-01 12:30:24,845 INFO mapreduce.JobSubmitter: Executing with tokens: []
2023-11-01 12:30:25,265 INFO conf.Configuration: resource-types.xml not found
2023-11-01 12:30:25,265 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2023-11-01 12:30:25,930 INFO impl.YarnClientImpl: Submitted application application_1698821571744_0001
2023-11-01 12:30:26,004 INFO mapreduce.Job: The url to track the job: http://Azhar:8088/proxy/application_16988
21571744_0001/
2023-11-01 12:30:26,008 INFO mapreduce.Job: Running job: job_1698821571744_0001
2023-11-01 12:30:40,397 INFO mapreduce.Job: Job job_1698821571744_0001 running in uber mode : false
2023-11-01 12:30:40,399 INFO mapreduce.Job:  map 0% reduce 0%
2023-11-01 12:30:51,769 INFO mapreduce.Job:  map 100% reduce 0%
2023-11-01 12:31:00,913 INFO mapreduce.Job:  map 100% reduce 100%
2023-11-01 12:31:01,959 INFO mapreduce.Job: Job job_1698821571744_0001 completed successfully
2023-11-01 12:31:02,167 INFO mapreduce.Job: Counters: 54
```

STEP 16: At last Print your output for the WordCount text file.

Using the Command : **hadoop fs -cat /outputword/***

OUTPUT:



RESULT: Thus the program to run a basic wordcount mapreduce program to understand mapreduce is excecuted and output is verified successfully.

BIG DATA ANALYTICS LAB

Ex. NO: 4

EXPERIMENT: 04

Install, configure and run python, numpy and pandas.

PROGRAM:

AIM: To Installing and Running Applications On python, numpy and pandas.

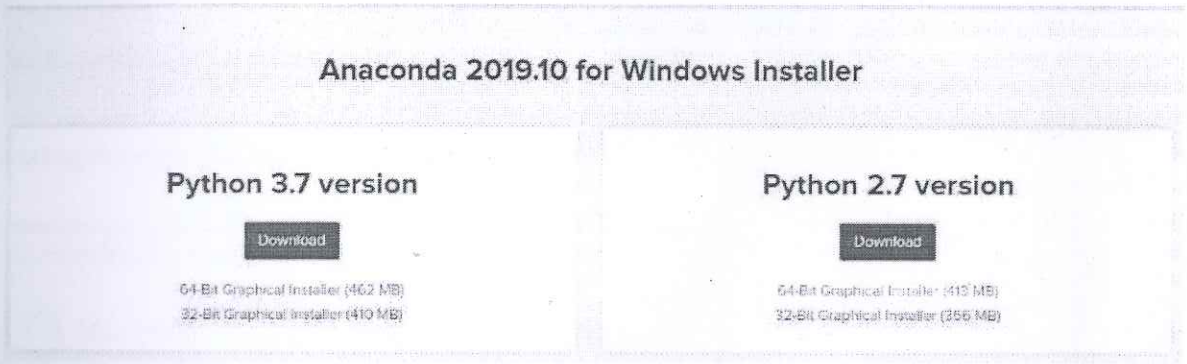
How to Install Anaconda on Windows?

Anaconda is an open-source software that contains Jupyter, spyder, etc that are used for large data processing, data analytics, heavy scientific computing. Anaconda works for R and python programming language. Spyder(sub-application of Anaconda) is used for python. Opencv for python will work in spyder. Package versions are managed by the package management system called conda.

To begin working with Anaconda, one must get it installed first. Follow the below instructions to Download and install Anaconda on your system:

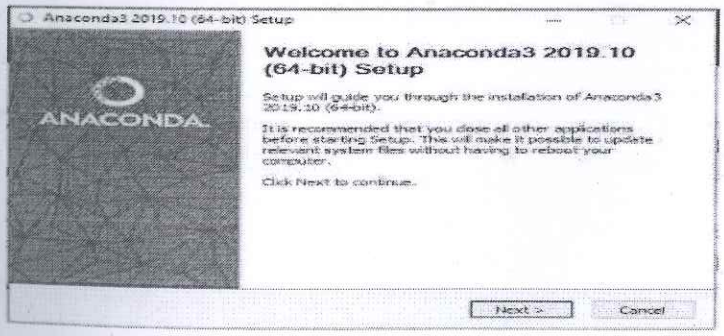
Download and install Anaconda:

Head over to anaconda.com and install the latest version of Anaconda. Make sure to download the -Python 3.7 Version for the appropriate architecture.



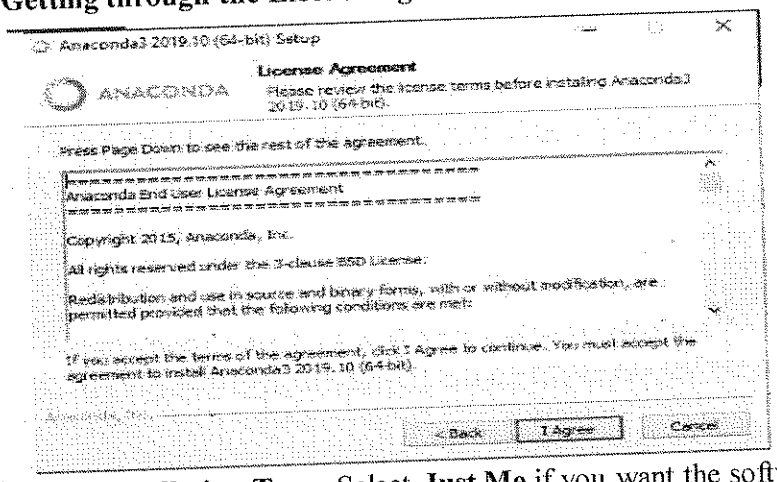
Begin with the installation process:

- **Getting Started**

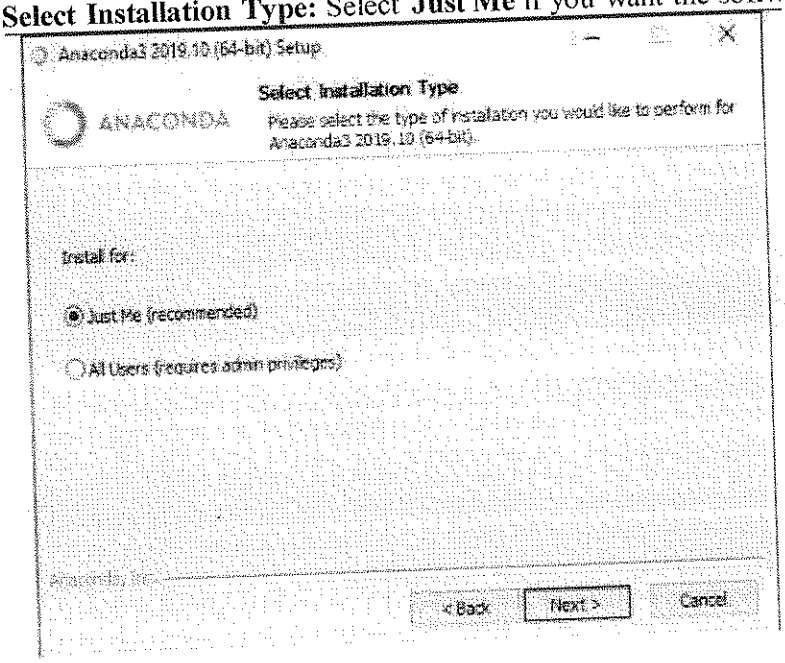


BIG DATA ANALYTICS LAB

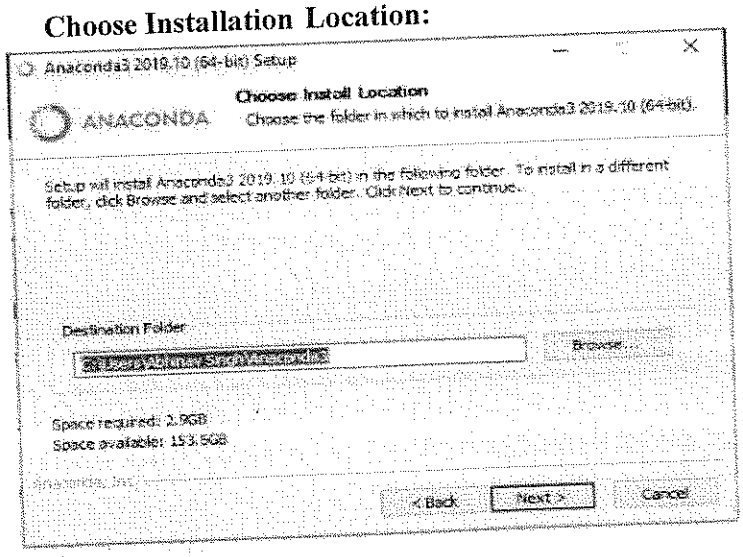
Getting through the License Agreement:



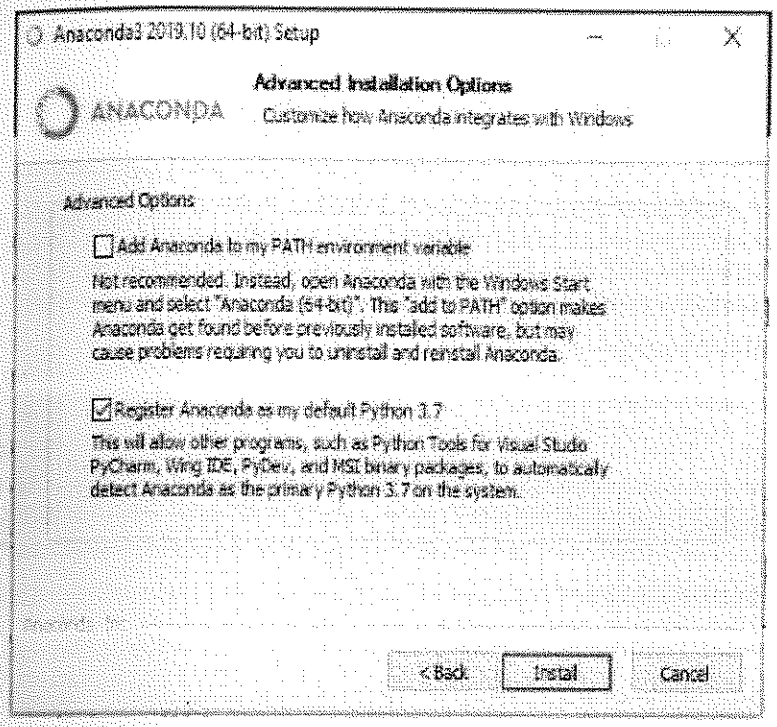
Select Installation Type: Select **Just Me** if you want the software to be used by a single User



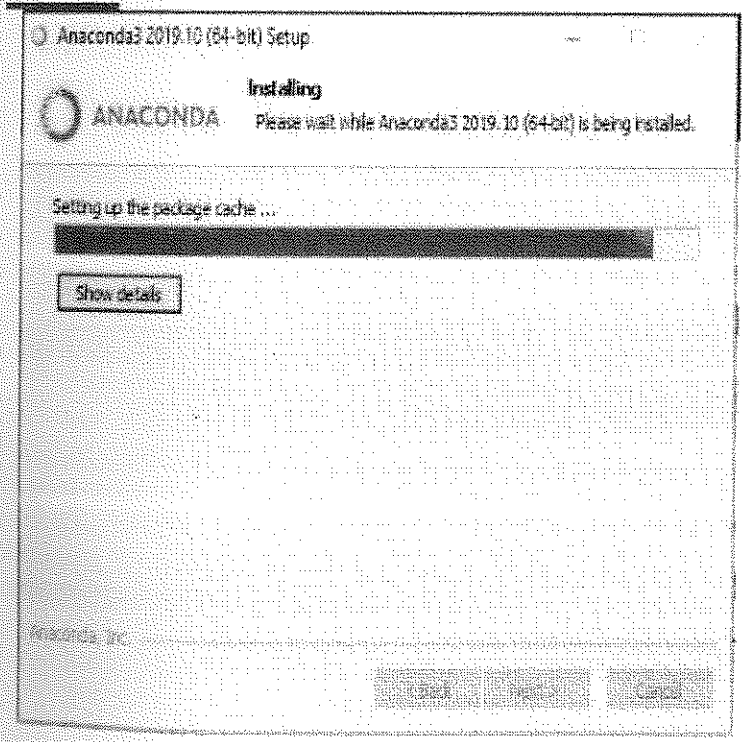
Choose Installation Location:



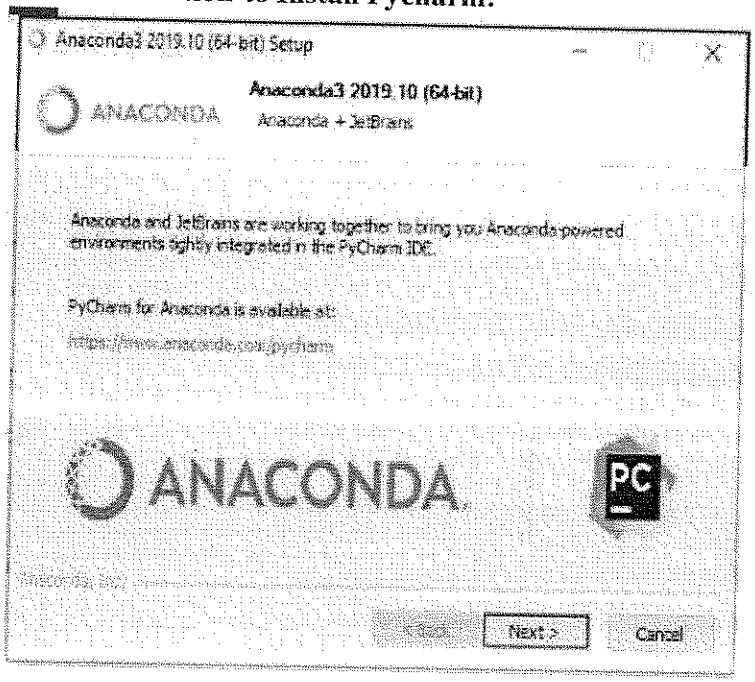
Advanced Installation Option:



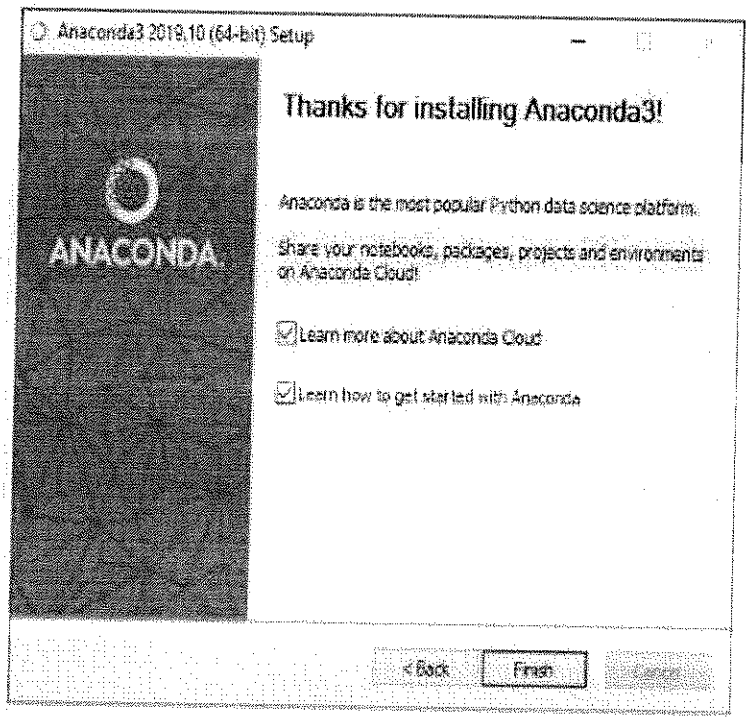
Getting through the Installation Process:



Recommendation to Install Pycharm:

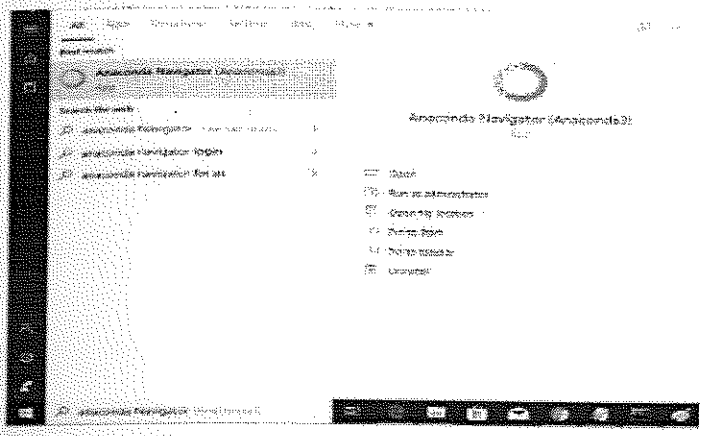


Finishing up the Installation:

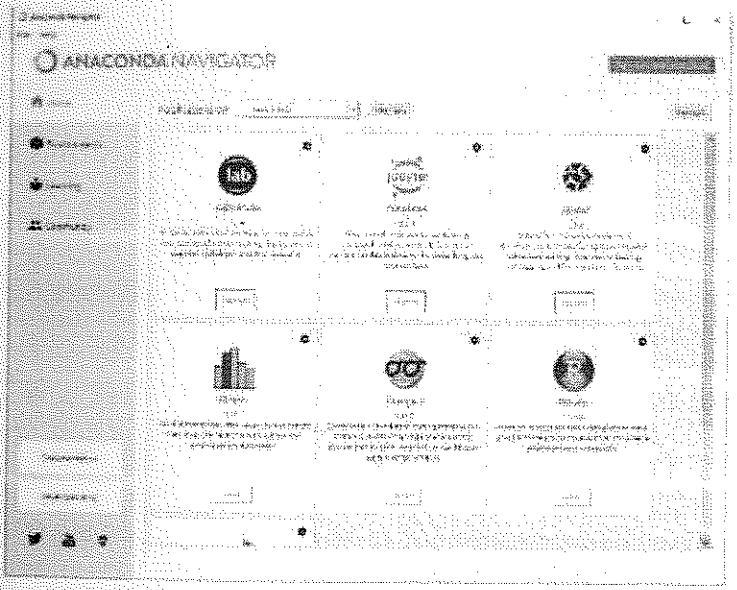


Working with Anaconda:

Once the installation process is done, Anaconda can be used to perform multiple operations. To



begin using Anaconda,search for
Anaconda Navigator from the Start Menu in Windows




```
#import pandas in jupyter notebook
import pandas

#loading the dataset which is excel file
dataset = pandas.read_csv("crime.csv")
```

#displaying the data
dataset

	Year	Population	Murder	Rape	Robbery	Assault	Burglary	CarTheft
0	1965	18073000	836	2320	28182	27464	183443	58452
1	1966	18258000	882	2439	30098	29142	196127	64368
2	1967	18336000	996	2665	40202	31261	219157	83775
3	1968	18113000	1185	2527	59857	34946	250918	104877
4	1969	18321000	1324	2902	64754	36890	248477	115400
5	1970	18190740	1444	2875	81149	39145	267474	125674
6	1971	18391000	1823	3225	97682	42318	273704	127658
7	1972	18366000	2026	4199	86391	45926	239886	105081
8	1973	18265000	2040	4852	80795	47781	246246	112328
9	1974	18111000	1919	5240	86614	51454	271824	104095
10	1975	18120000	1996	5099	93499	54593	301996	116274
11	1976	18084000	1969	4663	95718	54638	318919	133504
12	1977	17924000	1919	5272	84703	57193	309735	133669
13	1978	17748000	1820	5168	83785	58484	292956	119264
14	1979	17649000	2092	5394	93471	60949	308302	124343
15	1980	17506590	2228	5405	112273	60329	360925	133041
16	1981	17594000	2166	5479	120344	60189	350422	136849
17	1982	17659000	2013	5159	107643	59818	295245	137880
18	1983	17667000	1958	5296	94783	59452	249115	127861
19	1984	17735000	1786	5599	89900	64872	222956	115392
20	1985	17783000	1683	5706	89706	68270	219633	106537
21	1986	17772000	1907	5415	91360	76528	217010	113247
22	1987	17825000	2016	5537	89721	82417	218826	125329
23	1988	17898000	2244	5479	97434	91239	218060	153898
24	1989	17950000	2246	5242	103983	91571	211130	171007
25	1990	17990455	2605	5368	112360	92105	208813	187591
26	1991	18058000	2571	5085	112342	90186	204499	181287
27	1992	18119000	2397	5152	108154	87608	193548	168922
28	1993	18197000	2420	5008	102122	85802	181709	151949
29	1994	18169000	2016	4700	86617	82100	164650	128873
30	1995	18136000	1550	4290	72492	74351	146562	102596
31	1996	18185000	1353	4174	61822	64857	129828	89900

import pandas as pd
dataset1 = pd.read_csv("crime.csv")
dataset1

	Year	Population	Murder	Rape	Robbery	Assault	Burglary	CarTheft
0	1965	18073000	836	2320	28182	27464	183443	58452
1	1966	18258000	882	2439	30098	29142	196127	64368
2	1967	18336000	996	2665	40202	31261	219157	83775
3	1968	18113000	1185	2527	59857	34946	250918	104877
4	1969	18321000	1324	2902	64754	36890	248477	115400
5	1970	18190740	1444	2875	81149	39145	267474	125674
6	1971	18391000	1823	3225	97682	42318	273704	127658
7	1972	18366000	2026	4199	86391	45926	239886	105081
8	1973	18265000	2040	4852	80795	47781	246246	112328
9	1974	18111000	1919	5240	86614	51454	271824	104095
10	1975	18120000	1996	5099	93499	54593	301996	116274
11	1976	18084000	1969	4663	95718	54638	318919	133504
12	1977	17924000	1919	5272	84703	57193	309735	133669
13	1978	17748000	1820	5168	83785	58484	292956	119264
14	1979	17649000	2092	5394	93471	60949	308302	124343
15	1980	17506590	2228	5405	112273	60329	360925	133041
16	1981	17594000	2166	5479	120344	60189	350422	136849
17	1982	17659000	2013	5159	107643	59818	295245	137880
18	1983	17667000	1958	5296	94783	59452	249115	127861
19	1984	17735000	1786	5599	89900	64872	222956	115392
20	1985	17783000	1683	5706	89706	68270	219633	106537
21	1986	17772000	1907	5415	91360	76528	217010	113247
22	1987	17825000	2016	5537	89721	82417	218826	125329
23	1988	17898000	2244	5479	97434	91239	218060	153898
24	1989	17950000	2246	5242	103983	91571	211130	171007
25	1990	17990455	2605	5368	112360	92105	208813	187591
26	1991	18058000	2571	5085	112342	90186	204499	181287
27	1992	18119000	2397	5152	108154	87608	193548	168922
28	1993	18197000	2420	5008	102122	85802	181709	151949
29	1994	18169000	2016	4700	86617	82100	164650	128873
30	1995	18136000	1550	4290	72492	74351	146562	102596
31	1996	18185000	1353	4174	61822	64857	129828	89900

dataset1.head()

	Year	Population	Murder	Rape	Robbery	Assault	Burglary	CarTheft
0	1965	18073000	836	2320	28182	27464	183443	58452
1	1966	18258000	882	2439	30098	29142	196127	64368
2	1967	18336000	996	2665	40202	31261	219157	83775
3	1968	18113000	1185	2527	59857	34946	250918	104877
4	1969	18321000	1324	2902	64754	36890	248477	115400

dataset1.tail()

	Year	Population	Murder	Rape	Robbery	Assault	Burglary	CarTheft
42	2007	19297729	801	2926	31094	45094	64857	28030
43	2008	19467789	836	2799	31789	42122	65537	25096
44	2009	19541453	781	2582	28141	43606	62769	21871
45	2010	19395206	868	2797	28630	44197	65839	20639
46	2011	19465197	774	2752	28396	45568	65397	19311

dataset1.head(10)

	Year	Population	Murder	Rape	Robbery	Assault	Burglary	CarTheft
0	1965	18073000	836	2320	28182	27464	183443	58452
1	1966	18258000	882	2439	30098	29142	196127	64368
2	1967	18336000	996	2665	40202	31261	219157	83775
3	1968	18113000	1185	2527	59857	34946	250918	104877
4	1969	18321000	1324	2902	64754	36890	248477	115400
5	1970	18190740	1444	2875	81149	39145	267474	125674
6	1971	18391000	1823	3225	97682	42318	273704	127658
7	1972	18366000	2026	4199	86391	45926	239886	105081
8	1973	18265000	2040	4852	80795	47781	246246	112328
9	1974	18111000	1919	5240	86814	51454	271824	104095

dataset1.tail(10)

	Year	Population	Murder	Rape	Robbery	Assault	Burglary	CarTheft
37	2002	19134293	909	3885	36653	53583	76700	47366
38	2003	19212425	934	3775	35790	48987	75453	45204
39	2004	19280727	889	3608	33506	46911	70696	41002
40	2005	19315721	874	3636	35179	46150	68034	35736
41	2006	19306183	921	3169	34489	45387	68565	32134
42	2007	19297729	801	2926	31094	45094	64857	28030
43	2008	19467789	836	2799	31789	42122	65537	25096
44	2009	19541453	781	2582	28141	43606	62769	21871
45	2010	19395206	868	2797	28630	44197	65839	20639
46	2011	19465197	774	2752	28396	45568	65397	19311

type(dataset1)

pandas.core.frame.DataFrame

```
pandas.core.frame.DataFrame
```

#to find any null values in the last 5 rows

```
dataset1.isnull().tail()
```

	Year	Population	Murder	Rape	Robbery	Assault	Burglary	CarTheft
42	False	False	False	False	False	False	False	False
43	False	False	False	False	False	False	False	False
44	False	False	False	False	False	False	False	False
45	False	False	False	False	False	False	False	False
46	False	False	False	False	False	False	False	False

#to makesure that no null values exists

```
dataset1.notnull().tail()
```

	Year	Population	Murder	Rape	Robbery	Assault	Burglary	CarTheft
42	True	True	True	True	True	True	True	True
43	True	True	True	True	True	True	True	True
44	True	True	True	True	True	True	True	True
45	True	True	True	True	True	True	True	True
46	True	True	True	True	True	True	True	True

#displays the number of null values in each column

```
dataset1.isnull().sum()
```

```
Year          0
Population    0
Murder        0
Rape          0
Robbery       0
Assault       0
Burglary      0
CarTheft      0
dtype: int64
```

#helps to find null values with respect to ROBBERY column

```
dataset1[dataset1.Robbery.isnull()]
```

	Year	Population	Murder	Rape	Robbery	Assault	Burglary	CarTheft

```
dataset1.shape
```

```
(47, 8)
```

#helps to find how many times values in a particular column has repeated

```
dataset1['Robbery'].value_counts()
```

```
94783     1
34489     1
86814     1
86617     1
80795     1
97434     1
108154    1
120344    1
56094     1
28182     1
31094     1
30098     1
91360     1
59857     1
35790     1
36555     1
40202     1
83783     1
```

```
#consolidated value counts for all the columns in the dataset
for col in dataset1.columns:
    display(dataset1[col].value_counts())
```

```
1983    1
1995    1
2004    1
2003    1
2002    1
2001    1
2000    1
1999    1
1998    1
1997    1
1996    1
1994    1
2006    1
1993    1
1992    1
1991    1
1990    1
1989    1
1988    1
```

```
#helps to find number of rows in the dataset
dataset_length=len(dataset1) dataset_length
47
```

```
#helps to find number of columns in the dataset
dataset_col=len(dataset1.columns)
dataset_col
8
```

```
#helps to find the summary of numerical columns
dataset1.describe()
```

	Year	Population	Murder	Rape	Robbery	Assault	Burglary	CarTheft
count	47.000000	4.700000e+01	47.000000	47.000000	47.000000	47.000000	47.000000	47.000000
mean	1988.000000	1.834426e+07	1549.978723	4200.425532	70429.297872	58022.234043	189119.829787	97573.553191
std	13.711309	6.024504e+05	590.454265	1096.569507	30204.823764	17455.534367	90256.257143	46707.064488
min	1965.000000	1.750669e+07	774.000000	2320.000000	28141.000000	27464.000000	62769.000000	19311.000000
25%	1976.500000	1.793700e+07	922.500000	3197.000000	36804.000000	45477.500000	90581.500000	56246.000000
50%	1988.000000	1.816900e+07	1683.000000	4199.000000	81149.000000	57193.000000	208813.000000	106537.000000
75%	1999.500000	1.868373e+07	2016.000000	5241.000000	94141.000000	64864.500000	250016.500000	128367.000000
max	2011.000000	1.954145e+07	2605.000000	5706.000000	120344.000000	92105.000000	360925.000000	187591.000000

```
#helps to describe individual column
dataset1.Murder.describe()
```

```
count      47.000000
mean     1549.978723
std       590.454265
min       774.000000
25%       922.500000
50%      1683.000000
75%      2016.000000
max      2605.000000
Name: Murder, dtype: float64
```

```
dataset1.skew()
```

```
Year          0.000000
Population    0.795669
Murder        0.059733
Rape         -0.237130
Robbery      -0.134085
Assault       0.464637
Burglary     -0.020278
CarTheft     -0.129653
dtype: float64
```

```
dataset1.var()
```

```
Year          1.880000e+02
Population    3.629465e+11
Murder        3.486362e+05
Rape          1.202465e+06
Robbery       9.123314e+08
Assault       3.046957e+08
Burglary      8.146192e+09
CarTheft      2.181550e+09
dtype: float64
```

```
dataset1.kurtosis()
```

```
Year         -1.200000
Population   -0.692220
Murder       -1.513564
Rape         -1.471445
Robbery      -1.527674
Assault      -0.482013
Burglary     -1.186281
CarTheft     -0.951036
dtype: float64
```

```
print(dataset1.dtypes)
```

```
Year          int64
Population    int64
Murder        int64
Rape          int64
Robbery       int64
Assault       int64
Burglary      int64
CarTheft      int64
dtype: object
```

NUMPY

Numpy is the core library for scientific and numerical computing in Python. It provides high performance multi dimensional array object and tools for working with arrays.
Numpy main object is the multidimensional array, it is a table of elements (usually numbers) all of the same type indexed by a positive integers.

In Numpy dimensions are called as axes.
Numpy is fast, convenient and occupies less memory when compared to python list.

```
import numpy
arr = numpy.array([1, 2, 3, 4, 5])
print(arr)
[1 2 3 4 5]
```

NumPy is usually imported under the np alias.

import numpy as np

Now the NumPy package can be referred to as np instead of numpy.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr)
[1 2 3 4 5]
```

Checking NumPy Version
The version string is stored under __version___attribute.

```
import numpy as np
print(np._version_)
1.18.1
```

Create a NumPy ndarray Object
NumPy is used to work with arrays. The arrayobject in NumPy is called ndarray.
We can create a NumPy ndarray object byusing the array() function.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr)
print(type(arr))

[1 2 3 4 5]
<class 'numpy.ndarray'>
```

type(): This built-in Python function tells us the type of the object passed to it. Like in above code it shows that arr is numpy.ndarray type.
To create an ndarray, we can pass a list, tuple or any array-like object into the array() method, and it will be converted into an ndarray:

Use a tuple to create a NumPy array:

```
import numpy as np
arr = np.array((1, 2, 3, 4, 5))
print(arr)
[1 2 3 4 5]
```

Dimensions in Arrays
A dimension in arrays is one level of arraydepth (nested arrays).

nested array: are arrays that have arrays as their elements.

0-D Arrays
0-D arrays, or Scalars, are the elements in an array. Each value in an array is a 0-D array.

```
#Create a 0-D array with value 42
import numpy as np
arr = np.array(42)
print(arr)
```

```
42
```

1-D Arrays
These are the most common and basic arrays.

```
#Create a 1-D array containing the values 1,2,3,4,5:
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr)
```

```
[1 2 3 4 5]
```

2-D Arrays
An array that has 1-D arrays as its elements is called a 2-D array.
These are often used to represent matrix or 2nd order tensors.

```
#Create a 2-D array containing two arrays with the values 1,2,3 and 4,5,6:
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr)
```

```
[[1 2 3]
 [4 5 6]]
```

3-D arrays
An array that has 2-D arrays (matrices) as its elements is called 3-D array.
These are often used to represent a 3rd order tensor.

```
#Create a 3-D array with two 2-D arrays, both containing two arrays with the values 1,2,3 and 4,5,6:
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
print(arr)
```

```
[[[1 2 3]
  [4 5 6]]

 [[1 2 3]
  [4 5 6]]]
```

Check Number of Dimensions?
NumPy Arrays provides the ndim attribute that returns an integer that tells us how many dimensions the array have.

```
#Check how manydimensions the arrays have:
import numpy as np
a = np.array(42)
b = np.array([1, 2, 3, 4, 5])
c = np.array([[1, 2, 3], [4, 5, 6]])
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
print(a.ndim)
print(b.ndim)
print(c.ndim)
print(d.ndim)
```

```
0
1
2
3
```

```
#Create an array with 5 dimensions and verify that it has 5 dimensions:
import numpy as np
arr = np.array([1, 2, 3, 4], ndmin=5)
print(arr)
print('number of dimensions :', arr.ndim)
```

```
[[[[[1 2 3 4]]]]]
number of dimensions : 5
```

NumPy Array Indexing

Access Array Elements

Array indexing is the same as accessing an array element.

You can access an array element byreferring to its index number.

The indexes in NumPy arrays start with 0, meaning that the first element has index 0, and the second has index 1 etc.

#Get the first element from the following array:

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr[0])
```

#Get the second element from the following array.

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr[1])
```

#Get third and fourth elements from the following array and add them.

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr[2] + arr[3])
```

```
1
2
7
```

Access 2-D Arrays

To access elements from 2-D arrays we can use comma separated integers representing the dimension and the index of the element.

Think of 2-D arrays like a table with rows and columns, where the dimension represents the row and the index represents the column.

```
#Access the element on the first row, second column:
import numpy as np
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
print('2nd element on 1st row: ', arr[0, 1])
```

```
2nd element on 1st row:  2
```

```
#Access the element on the 2nd row, 5th column:
import numpy as np
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
print('5th element on 2nd row: ', arr[1, 4])
```

```
5th element on 2nd row:  10
```

Result

Ex. NO: 5

EXPERIMENT: 05

Visualize Data Using Basic Plotting Techniques In Python.

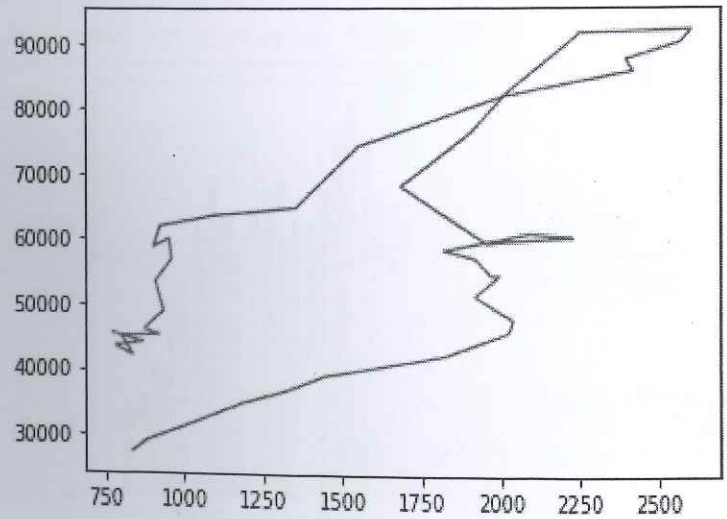
PROGRAM:

AIM: To create an application that takesthe Visualize Data Using Basic Plotting Techniques.

```
import pandas as pb
import matplotlib.pyplot as plt
import seaborn as sns
crime=pb.read_csv('crime.csv')
crime
```

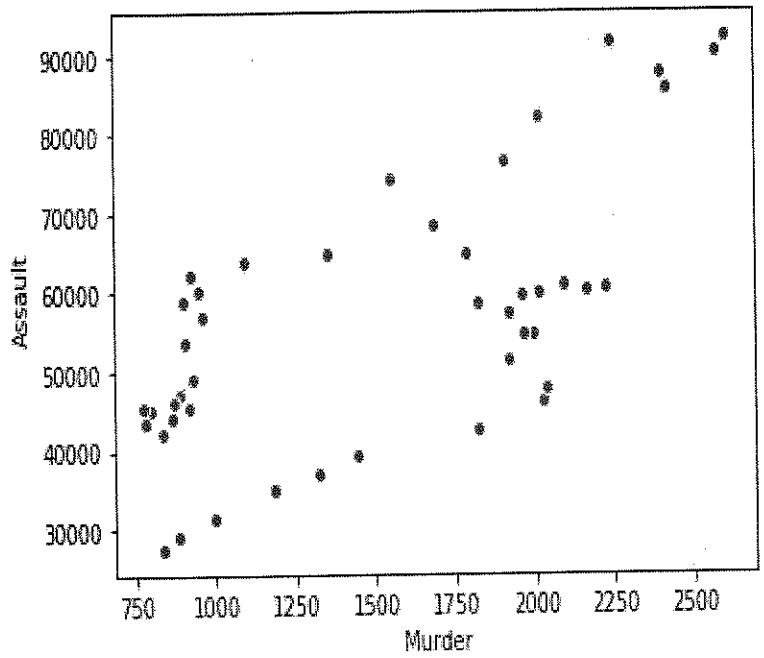
	Year	Population	Murder	Rape	Robbery	Assault	Burglary	CarTheft
0	1965	18073000	836	2320	28182	27464	183443	58452
1	1966	18258000	882	2439	30098	29142	196127	64368
2	1967	18336000	996	2665	40202	31261	219157	83775
3	1968	18113000	1185	2527	59857	34946	250918	104877
4	1969	18321000	1324	2902	64754	36890	248477	115400
5	1970	18190740	1444	2875	81149	39145	267474	125674
6	1971	18391000	1823	3225	97682	42318	273704	127658
7	1972	18366000	2026	4199	86391	45926	239886	105081
8	1973	18265000	2040	4852	80795	47781	246246	112328
9	1974	18111000	1919	5240	86814	51454	271824	104095
10	1975	18120000	1996	5099	93499	54593	301996	116274
11	1976	18084000	1969	4663	95718	54638	318919	133504
12	1977	17924000	1919	5272	84703	57193	309735	133669
13	1978	17748000	1820	5168	83785	58484	292956	119264
14	1979	17649000	2092	5394	93471	60949	308302	124343
15	1980	17506690	2228	5405	112273	60329	360925	133041
16	1981	17594000	2166	5479	120344	60189	350422	136849
17	1982	17659000	2013	5159	107843	59818	295245	137880
18	1983	17667000	1958	5296	94783	59452	249115	127861
19	1984	17735000	1786	5599	89900	64872	222956	115392
20	1985	17783000	1683	5706	89706	68270	219633	106537

```
plt.plot(crime.Murder,crime.Assault);
```

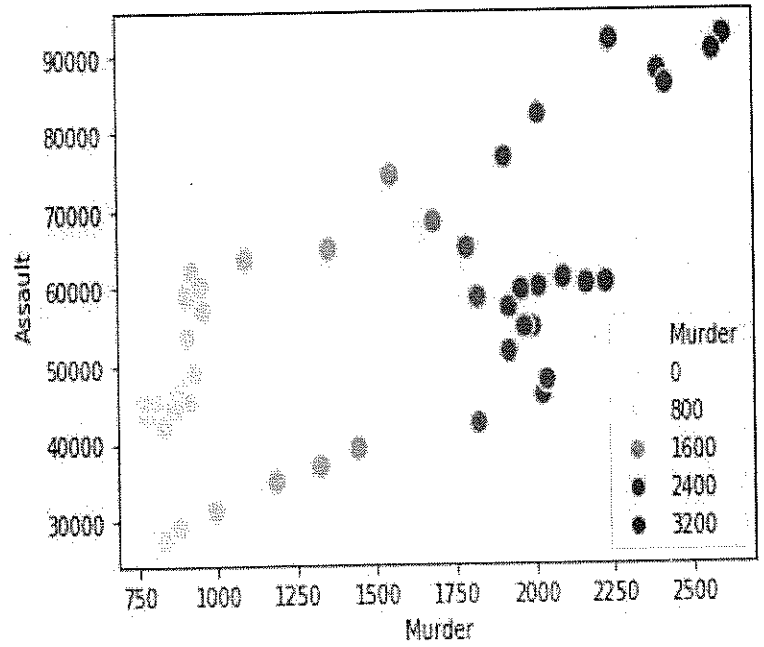



```
import seaborn as sns
```

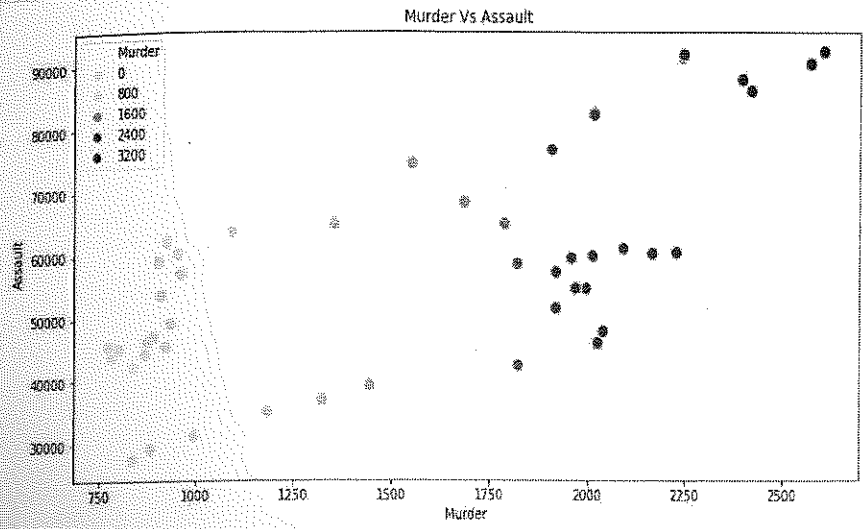
```
sns.scatterplot(crime.Murder,crime.Assault);
```



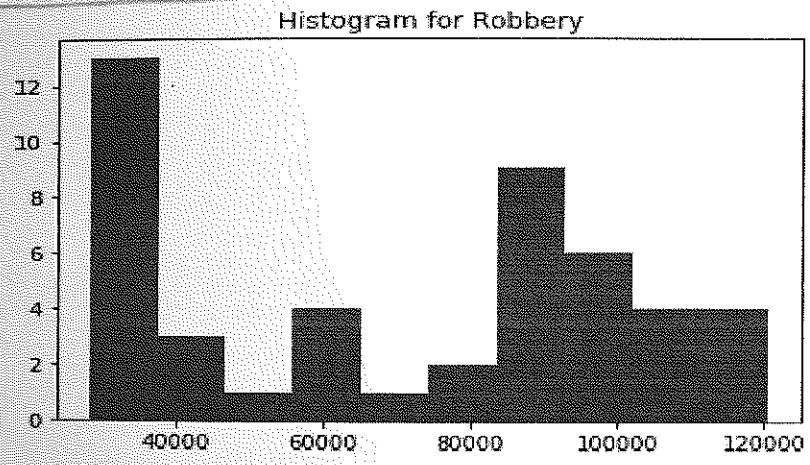

```
sns.scatterplot(crime.Murder,crime.Assault,hue=crime.Murder,s=100);
```



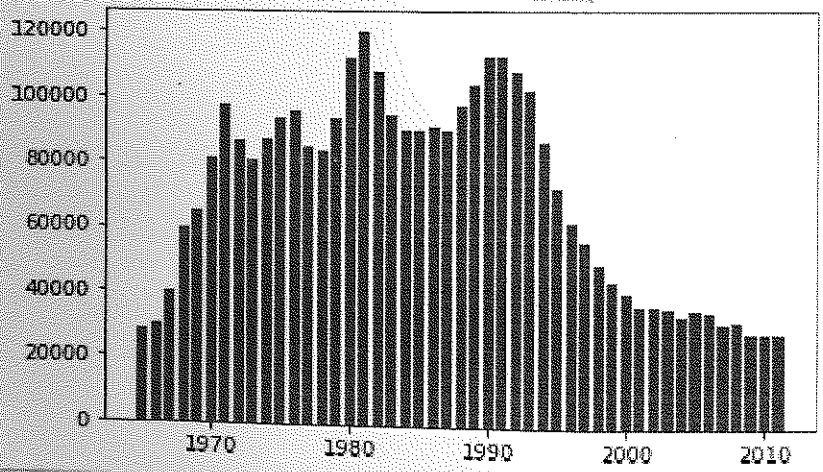

```
plt.figure(figsize=(12,6))
plt.title('Murder Vs Assault')
sns.scatterplot(crime.Murder,crime.Assault,hue=crime.Murder,s=100);
```



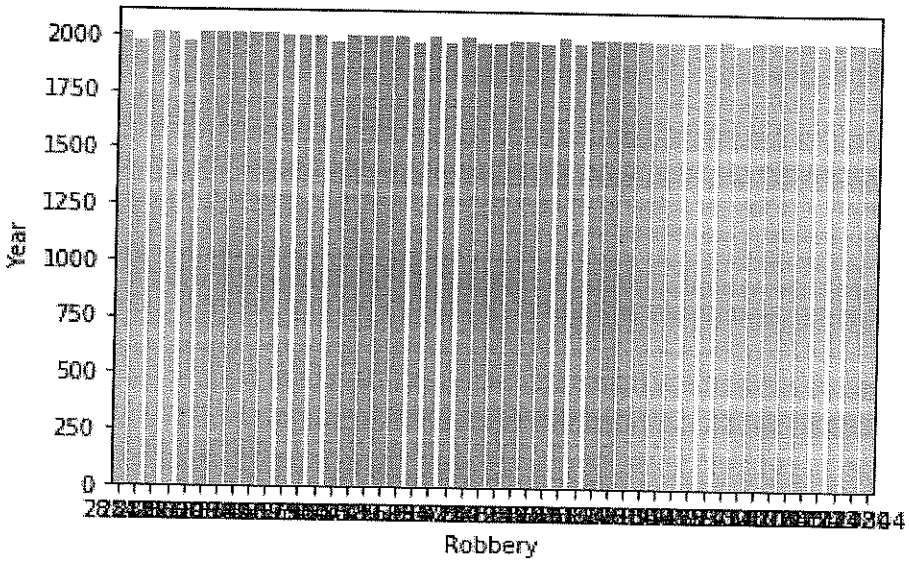

```
plt.title('Histogram for Robbery')
plt.hist(crime.Robbery);
```



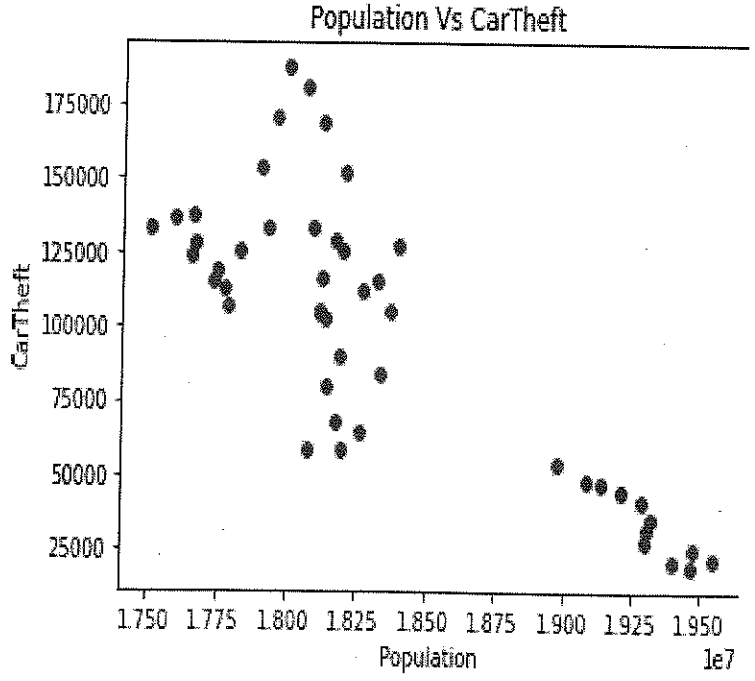

```
plt.bar(crime_bar.index,crime_bar.Robbery);
```




```
sns.barplot('Robbery','Year',data=crime);
```




```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
data=pd.read_csv('crime.csv')
x=data.Population
y=data.CarTheft
plt.scatter(x,y)
plt.xlabel('Population')
plt.ylabel('CarTheft')
plt.title('Population Vs CarTheft')
plt.show();
```



Result :